

Public Key Infrastructure

Prepared by

Jesna J S

Assistant Professor CSE, TKMCE

KEY DISTRIBUTION USING ASYMMETRIC ENCRYPTION

- In **cryptology**, two users must share a **secret key** before using **symmetric (conventional) encryption**.
- **Difficulty:**
 - ✓ If users are **far apart**, sharing the secret key securely is difficult.
 - ✓ Sending the key directly over the Internet is **unsafe**.
 - ✓ If every pair of users needs a separate key, the system becomes **complex and unmanageable**.
- **Asymmetric (public-key) encryption solves the key distribution problem.**
- Public-key encryption helps in **two ways**:
 1. **Distribution of public keys**
 2. **Secure distribution of secret (symmetric) keys**

- In public-key cryptography:
- Each user has:
 - A **Public Key** (shared with everyone)
 - A **Private Key** (kept secret)
- Anyone can **pretend to be someone else** and send a fake public key..
(**Public Key Forgery Attack.**).
- To solve false public key problems, we use **Public-Key Certificates**.

Public-Key Certificate

- It is a **digital document** that contains:
 - User's **identity**
 - User's **public key**
 - **Digital signature of a trusted authority**
- A trusted organization called a **Certificate Authority (CA)** will issues the certificates.
- Example: Government agency, Bank, DigiCert, etc.

Public Key Infrastructure

- Public key infrastructure or PKI is the governing body behind issuing digital certificates.
- It consists of
 1. Certificates
 2. Repository for retrieving certificates
 3. A method of revoking certificates
 4. A method of evaluating a chain of certificates from public keys that are known and trusted in advance (trust anchors) to the target name.

- The principal objective for developing a PKI is to **enable secure, convenient, and efficient acquisition of public keys**
- It helps to protect confidential data and gives unique identities to users and systems. Thus, **it ensures security in communications.**
- The public key infrastructure **uses a pair of keys: the public key and the private key to achieve security.**
- The public keys are prone to attacks and thus an intact infrastructure is needed to maintain them.

- A certificate authority (CA) issues digital certificates to users, devices, and institutions.
- A registration authority (RA) verifies the identity of the user or device requesting a certificate.
- A certificate database stores the digital certificate and its metadata.
- A validation authority (VA) verifies that a certificate has not been revoked.
- A recipient uses a private key to decrypt a message encrypted with a public key

PKI Trust Models

The **trust model** defines **how entities (users, devices, or organizations) establish trust in digital certificates issued within the PKI.**

1. Monopoly model
2. Monopoly plus registration authorities
3. Delegated CA
4. Oligarchy
5. Anarchy Model
6. Name constraints
7. Top down with name constraints
8. Bottom-Up PKI Model with Name Constraints

1. Monopoly Model

- A **Monopoly PKI Trust Model** is a hierarchical trust model where **a single, central Certificate Authority (CA)** is responsible for issuing, verifying, and managing all digital certificates within the system.
- This model assumes that every entity (user, device, or organization) **trusts** this single CA unconditionally.

Characteristics:

- ✓ Only one CA is responsible for issuing all certificates.
- ✓ All users and systems trust the same CA.
- ✓ Easier to manage compared to distributed models.
- ✓ If the CA is compromised, the entire trust system collapses.
- ✓ Difficult to scale globally as trust depends on a single entity.

2. Monopoly plus registration authorities

- It is an extension of the **Monopoly PKI Model**, where a **single Certificate Authority (CA)** is responsible for issuing certificates, **but it delegates the verification and authentication of users to multiple Registration Authorities (RAs).**

Characteristics

- Central Certificate Authority (CA):
 - The sole entity responsible for issuing, revoking, and managing digital certificates.
 - All trust in the system is placed in this CA.

- Multiple Registration Authorities (RAs):
 - ✓ RAs act as intermediaries between users and the CA.
 - ✓ They handle identity verification before forwarding certificate requests to the CA.
 - ✓ The CA relies on the RAs' validation process to approve or reject certificate issuance.
- Users & Organizations:
 - ✓ They request digital certificates from RAs, which then communicate with the CA.
 - ✓ Once validated, the CA issues the certificate.

Advantages:

- 1. Increased Scalability:** RAs help distribute the workload of identity verification.
- 2. Enhanced Security:** The CA does not handle direct authentication, reducing attack vectors.
- 3. Delegation of Responsibilities:** Organizations can have local RAs while still trusting a central CA.

Disadvantages:

- 1. Single Point of Failure (CA):** If the CA is compromised, the entire PKI structure is affected.
- 2. Trust in RAs:** If an RA is compromised, it can submit fraudulent requests to the CA.
- 3. Operational Complexity:** Requires secure communication and trust management between CA and RAs.

3. Delegated CA

- The **Delegated CA PKI Model** is an extension of the traditional **hierarchical (tree-based) PKI model**, where the **Root Certificate Authority (Root CA) delegates the responsibility of issuing certificates to one or more subordinate CAs**.
- This delegation allows for better scalability, security, and management of digital certificates across different organizations or regions.

Characteristics

- **Root CA:**
 - The highest trust authority in the PKI hierarchy.
 - Issues certificates **only to Subordinate CAs** (Delegated CAs), not directly to end-users.
 - Maintains strict security and is often kept **offline** to prevent compromise.

- **Delegated (Subordinate) CAs:**
 - Receive certificates from the Root CA and **act as intermediate CAs.**
 - Responsible for issuing and managing certificates for end-users, devices, or organizations.
 - Can be specific to departments, business units, or geographic locations.
- **End Users / Entities:**
 - These are the recipients of certificates issued by **Delegated CAs.**
 - Can include individuals, web servers, email systems, or software applications.

How it works?

- The Root CA issues certificates only to Delegated CAs.
- The Delegated CA receives its certificate and is trusted as an authority.
- The Delegated CA issues certificates to users, servers, or applications.
- When verifying a certificate, the trust chain is checked from Delegated CA $\rightarrow$ Root CA.

Advantages:

- ✓ **Scalability:** Can handle a large number of users by distributing responsibility.
- ✓ **Security:** The Root CA is **offline**, reducing the risk of compromise.
- ✓ **Operational Efficiency:** Different Delegated CAs can serve different functions.
- ✓ **Granular Trust Control:** Each CA can enforce its own security policies

Disadvantages:

Complex Management: More CAs mean more security policies and trust relationships to manage.

Risk of Compromised Delegated CAs: If a Delegated CA is compromised, all its issued certificates become untrustworthy.

Certificate Chain Complexity: Verification requires checking multiple certificate levels (Delegated CA → Root CA).

4. Oligarchy PKI Trust Model

- The **Oligarchy PKI Model** is a trust model where **multiple Certificate Authorities (CAs) share control** over issuing and verifying digital certificates.
- Instead of a single monopoly CA (as in the **Monopoly PKI Model**), multiple trusted CAs coexist and are recognized by users, applications, and systems

Characteristics

- **Multiple Trusted CAs:**
 - Several independent CAs exist, and each is trusted equally within the system.
 - Organizations and users can obtain certificates from any of these CAs

- **Distributed Trust Management:**
 - Unlike a single CA monopoly, multiple CAs **reduce the risk of a single point of failure.**
 - However, trust still remains **centralized** within a small group of CAs.
- **Common in Public PKI:**
 - The Oligarchy model is often used in **web browsers, email encryption, and SSL/TLS certificates.**
 - Browsers maintain a **list of trusted CAs** (e.g., DigiCert, GlobalSign, Sectigo, Let's Encrypt, etc.).
- **Certificate Interoperability:**
 - *Certificates issued by any of the recognized CAs are valid and can be used across different platforms.*
 - This enables widespread adoption in **internet security, banking, and corporate authentication.**

- Google Chrome, Mozilla Firefox, and Microsoft Edge trust multiple CAs.
- Websites can obtain an SSL/TLS certificate from any recognized CA (e.g., DigiCert, Let's Encrypt, Sectigo).
- If one CA is compromised, the browser can remove it from the trust list without affecting the others.

Advantages

- **No Single Point of Failure:** If one CA is compromised, others remain operational.
- **Greater Flexibility:** Users and organizations can choose **different CAs** based on cost, security policies, or geographic preference.
- **Better Interoperability:** Certificates from multiple trusted CAs work seamlessly across different platforms and application

Disadvantages

- **Trust Fragmentation:** Different organizations or systems may trust **different sets of CAs**, leading to **compatibility issues**.
- **CA Compromise Risk:** If a **trusted CA is hacked**, attackers can issue fraudulent certificates, impacting multiple systems.
- **Revocation Challenges:** If a CA is **removed from the trust list**, all its previously issued certificates become invalid, affecting users globally

5. Anarchy Model

- The **Anarchy PKI Model**, also known as the **Web of Trust (WoT) Model**, is a decentralized trust model where there is **no central Certificate Authority (CA)** controlling certificate issuance or validation.
- Instead, trust is established **peer-to-peer**, where individuals or organizations directly sign each other's certificates.

Characteristics:

- **No Central Authority (CA):**
- Unlike the **Monopoly or Oligarchy PKI Models**, there is no single CA issuing certificates.
- **Each entity (user or organization) is responsible for validating and signing certificates.**

- **Trust is Built Peer-to-Peer:**

- Users verify and **digitally sign** each other's certificates based on personal or organizational trust.
- The more trusted signatures a certificate has, the more credibility it gains.

- **Decentralized and Self-Regulated:**

- Unlike **traditional hierarchical PKI**, there's no single point of failure.
- Users decide **whom to trust**, reducing reliance on third parties.

- **Common in Open-Source & Secure Communication**

Example: Pretty Good Privacy for Email

1. Alice wants to send Bob an encrypted email.
2. Instead of relying on a CA, Bob gives Alice his public key directly.
3. Alice can verify Bob's key by checking if other trusted people have signed it.
4. If enough trusted individuals vouch for Bob's key, Alice accepts it as authentic.

Advantages

- **No Single Point of Failure:** Trust is distributed, making it harder to attack.
- **More User Control:** Individuals decide whom they trust instead of relying on a CA.
- **Decentralized & Scalable:** Works well in open, global systems like secure communication and cryptocurrency.

Disadvantages

- **Difficult Trust Management:** Users must manually verify and sign others' certificates.
- **No Universal Trust:** Different people trust different entities, leading to fragmented trust.
- **Less Automated Verification:** Unlike CA-based models, there is no centralized revocation system if a key is compromised.

6. Name Constraints PKI Model

- The **Name Constraints PKI Model** is a specialized **Public Key Infrastructure (PKI) trust model** that **restricts** the domains, email addresses, or IP addresses for which a Certificate Authority (CA) can issue certificates.
- This model is primarily used to **enhance security** by ensuring that a CA cannot issue certificates outside of its authorized namespace.
- **A CA can only issue certificates for pre-defined domains, subdomains, or IP ranges.**
- *Example: A CA with a constraint on *.example.com can issue certificates only for example.com and its subdomains but not for otherdomain.com.*
- Prevents Certificate Misuse & Misissuance

- **Advantage:**

1. **Prevents Unauthorized Certificate Issuance:** Ensures CAs can issue certificates **only for authorized domains or namespaces**.
2. **Enhances Security:** Reduces the impact of **CA compromise or misconfiguration**.
3. **Supports Delegated CA Management:** Large organizations can enforce trust boundaries within their PKI infrastructure.

7. Top-Down PKI Model with Name Constraints

- The **Top-Down PKI Model with Name Constraints** combines a **hierarchical (top-down) Public Key Infrastructure (PKI) structure with Name Constraints**, ensuring that subordinate Certificate Authorities (CAs) can **only issue certificates within a restricted namespace**.
- This model **enhances security** by preventing unauthorized certificate issuance, while still allowing a **centralized Root CA** to delegate certificate issuance responsibilities to Subordinate CAs.

8. Bottom-Up PKI Model with Name Constraints

- The **Bottom-Up PKI Model with Name Constraints** is a **decentralized approach** to Public Key Infrastructure (PKI) **where multiple independent entities (subordinate CAs) operate autonomously** but eventually link to a higher-level authority (Root CA).
- **Name Constraints** are applied to limit the authority of each entity, preventing them from issuing certificates beyond their defined namespace.
- **Multiple independent Subordinate CAs** are initially set up and operate separately.
- Later, they establish a trust relationship with a higher-level **Root CA**.

- Each Subordinate CA is allowed to issue certificates only within a specific namespace (e.g., a domain, IP range, or email domain).
- This prevents a CA from issuing unauthorized certificates for other domains.
- Unlike a **Top-Down PKI**, where trust flows from the Root CA to Subordinate CAs, here, Subordinate CAs operate independently until they are **recognized** by a Root CA.
- Suitable for **multi-organization networks**, where different entities operate their own CAs but later integrate into a centralized trust system.
- Greater Autonomy for Subordinate CAs:

What happens when a browser wants to communicate to a website?

1. Visiting a Secure Website (HTTPS)

- You type `https://www.example.com` in your browser.
- The website presents its SSL/TLS certificate to your browser.
- Your browser checks the certificate chain before trusting the website.

2. Steps of Certificate Verification:

- Check the Website's SSL/TLS Certificate
- The browser checks if the certificate is valid (not expired and properly signed).

3. Verify the Intermediate CA

- The website's certificate is issued by an Intermediate CA.
- The browser checks if the Intermediate CA is trusted.

4. Verify the Root CA .
 - The Intermediate CA's certificate is issued by a Root CA.
 - The browser checks if the Root CA is in its trusted root store.
5. Check for Revocation
 - The browser checks if the certificate is revoked.
 - If revoked, the website is not trusted.
6. If Everything is Valid, the Connection is Secured
 - The browser establishes an encrypted connection using SSL/TLS.



REVOCATION

Revocation

- Certificate revocation is the process of **invalidating a digital certificate before its natural expiration date**. Once revoked, the certificate is no longer **trusted** and should not be used for secure communications.
- Reasons for Revoking a Digital Certificate
 1. **Key Compromise**: If the **private key** associated with the certificate is compromised (e.g., stolen, leaked, or hacked).
 2. **Certificate Holder No Longer Trusted** : Example: A company revokes a certificate after an employee leaves the organization.
 3. **Certificate Misuse**: If a certificate is being used for fraudulent or unauthorized purposes

4. **Change in Business Operations**: Example: A website or company **changes its domain name or ownership**, requiring a new certificate.
5. **Incorrect Information**: If a **certificate contains incorrect details** (e.g., wrong domain name or owner details).
6. **CA Compromise**: If the Certificate Authority itself is compromised, all certificates issued by it may need to be revoked.

Revocation Mechanism:

1. **Delta CRLs**
2. **First Valid Certificate**
3. **OLRS Schemes**
4. **Good-lists vs Bad-lists**

1. Delta CRLs:

- A **Delta Certificate Revocation List (Delta CRL)** is a smaller, **incremental list** that contains **only the certificates revoked since the last full CRL was published**.
- **Instead of downloading a large full CRL every time, clients can download a Delta CRL to quickly update their revocation information.**

Why Delta CRLs:

- Full CRLs can become **very large** as more certificates are revoked, making downloads **slow and inefficient**.
- Delta CRLs **reduce bandwidth usage** and improve efficiency by **sending only recent changes**.
- Useful for systems where **real-time revocation checking** (like OCSP) is not feasible.

How Delta CRLs Work?

1. The Certificate Authority (CA) periodically publishes a Full CRL (e.g., once a week).
2. The CA also generates Delta CRLs (e.g., every hour or day) containing only new revocations.
3. Clients that already have the last Full CRL only need to download the Delta CRL to stay updated.
4. Clients can combine the Full CRL + Delta CRL to get the latest revocation status.

Advantages of Delta CRLs

- **Efficient & Saves Bandwidth** – Clients don't need to repeatedly download large CRLs.
- **Faster Updates** – Only the latest revocations are downloaded, making updates quick.
- **Better for Large-Scale Deployments** – Works well in **enterprise PKI** and **IoT** where thousands of certificates exist.

Disadvantages of Delta CRLs

- **Clients Must Handle Both Full & Delta CRLs** – Requires extra logic to merge them.
- **Not as Real-Time as OCSP** – Revocation is only checked when a client downloads the CRL.
- **Complexity for CAs** – The CA must manage **both Full CRLs and Delta CRLs** properly.

2. FIRST VALID CERTIFICATE

- The **First Valid Certificate** concept in PKI revocation mechanisms is primarily associated with **certificate path validation** and **revocation checking**.
- It ensures that a **valid, non-revoked certificate** is found within a **certificate chain** before trust is established..
- To validate a certificate, the system checks:
 - ✓ **Is the certificate valid (not expired)?**
 - ✓ **Is it properly signed by a trusted CA?**
 - ✓ **Is it revoked? (via CRL or OCSP).**

What is the "First Valid Certificate" Concept?

- When verifying a certificate chain, the system **checks each certificate in order** (from end-entity to root).
- **The first valid certificate is the first certificate in the chain that is not revoked, correctly signed, and valid.**
- If this certificate is found, the system **stops checking further up the chain** and trusts the certificate.
- **If it is revoked, the system continues checking until it finds a valid one or rejects the chain.**

Significance of the First Valid Certificate Concept

- **Optimizes revocation checks** – Saves time by stopping at the first trusted certificate.
- **Ensures security** – Prevents trusting a revoked certificate.
- **Improves performance** – Reduces unnecessary revocation lookups.

3. OLRs (On-Line Revocation Server) Scheme

- The **On-Line Revocation Server (OLRS)** scheme is a method used in **Public Key Infrastructure (PKI)** to provide **real-time certificate revocation status** without relying on large revocation lists like CRLs.
- It is an alternative to **Certificate Revocation Lists (CRLs)**

Key Features of OLRs

- ©**Real-time revocation checking** – Unlike CRLs, which are periodically updated, OLRs provide instant status verification.
- © **Efficient and lightweight** – Instead of downloading a full CRL, clients query only for the certificate in question.
- ©**Reduces latency** – Especially useful for high-security environments like financial transactions, VPNs, and secure web browsing.

How OLRS Works in PKI Revocation

1. Certificate Status Request:

- A client (e.g., web browser, VPN, or email system) needs to verify whether a digital certificate is revoked or valid.
- Instead of checking an entire CRL (Certificate Revocation List), it queries an OLRS server directly.

2. OLRS Server Query

- The OLRS acts as a centralized revocation status provider.
- It receives the request and checks its revocation database.

3. Real-Time Response:

- The OLRS **immediately** sends back the status (Valid, Revoked, or Unknown).
- This **response is signed to ensure authenticity**.

4. Decision:

- If the certificate is **valid**, the system proceeds with communication.
- If **revoked**, the system **rejects the connection**.

4. Good Lists vs Bad lists

- Two primary approaches to revocation checking are **Good-Lists** and **Bad-Lists**.
 1. Bad-Lists Revocation Scheme (Traditional Approach)
 2. Good-Lists Revocation Scheme (Modern Approach)

(1)Bad-Lists Revocation Scheme

⌘ A **Bad-List** contains a **list of revoked or untrusted certificates**. This is the **traditional** revocation approach and is used in:

- Certificate Revocation Lists (CRLs)
- Online Certificate Status Protocol (OCSP)

⌘ **How Bad-Lists Revocation scheme works?**

- The **Certificate Authority (CA)** or a revocation server maintains a list of **revoked certificates**.
- When a system (e.g., a browser) encounters a certificate, it checks if the certificate appears on the **Bad-List**.
- If the certificate is **on the list**, it is considered **revoked** and rejected
- If the certificate is **not on the list**, it is assumed to be **valid**.

Problems with Bad-Lists:

1. **Slow and inefficient** – Systems must download and search through large CRLs.
2. **High overhead** – Frequent updates are needed for real-time security
3. **Scalability issues** – Large networks and frequent revocations make bad-lists impractical.

(2) Good-Lists Revocation Scheme

- A **Good-List** maintains a **list of valid and trusted certificates**, rather than listing revoked ones.
- **How It Works:**
 - ✓ The **CA or a trusted authority** maintains a list of **valid certificates** (instead of revoked ones).
 - ✓ When a system needs to verify a certificate, it checks whether it appears in the **Good-List**.
 - ✓ If the certificate is **on the Good-List**, it is considered **valid and trusted**.
 - ✓ If the certificate is **not on the list**, it is assumed to be **revoked, untrusted, or unknown**.

- **Advantages:**

- ✓ **Faster verification** – Systems only need to check a small, trusted list instead of searching large CRLs.
- ✓ **Less overhead** – No need to continuously update long lists of revoked certificates.
- ✓ **More secure** – If a certificate is missing from the list, it is automatically untrusted.

- **Disadvantages:**

- ✓ **Requires frequent updates** – The Good-List must be updated regularly as new certificates are issued.
- ✓ **Storage overhead** – Large PKI systems need to maintain a list of all valid certificates